

Real-Time Event-Driven Notification System

Fault-tolerant distributed messaging with WebSockets & AWS SNS/SQS

Tech Stack | Node.js · WebSockets · AWS SNS/SQS · MongoDB · React.js · CloudWatch

Distributed Systems	Event-Driven	AWS	Real-Time	Microservices
---------------------	--------------	-----	-----------	---------------

Project Overview

- A production-grade, fault-tolerant distributed notification system built on an event-driven architecture. The system leverages WebSockets for persistent, low-latency client connections and AWS SNS/SQS for asynchronous, decoupled message routing across stateless microservice modules — ensuring reliable delivery at scale.

Key Technical Highlights

- Persistent WebSocket connections for real-time bi-directional client-server communication with minimal latency.
- AWS SNS for fan-out pub/sub messaging and SQS as a buffer queue between producers and consumers, decoupling components completely.
- Zero message loss guarantee under traffic spikes via SQS buffering, exponential backoff retry strategy, and dead-letter queues (DLQs) for failed messages.
- CloudWatch alarms integrated for real-time production observability — alerting on queue depth, error rates, and latency thresholds.
- Stateless service layer deployed behind a load balancer, enabling horizontal scaling of each microservice independently.

Architecture & Design Decisions

- Clients connect via WebSocket gateway; events published to SNS topics which fan out to SQS subscriber queues.
- Worker services consume from SQS queues and process notifications asynchronously — fully decoupled from the producer.
- Dead-letter queues capture failed messages after max retries; CloudWatch monitors DLQ depth to trigger alerts.
- MongoDB stores notification state and delivery receipts with indexed queries for fast per-user history retrieval.
- Horizontal scaling achieved by adding stateless worker instances behind the load balancer without any downtime.

Outcomes & Impact

- Achieved zero message loss under simulated traffic spike scenarios via queue buffering and retry logic.
- Real-time observability dashboard through CloudWatch enabled proactive detection of bottlenecks before user impact.
- Architecture supports independent scaling of each microservice component — designed for production-grade growth.
- Demonstrated strong command of distributed systems design patterns: pub/sub, event-driven architecture, and fault tolerance.